学习模型： 引入一个虚拟的、好学的学生“小明”，跟随他的视角探索游戏引擎的构建过程。

---

2. 游戏引擎的五层架构概述 *Content-[02:03]

一个现代游戏引擎，从用户视角到硬件底层，可以被抽象为五个核心层次。

1. **工具层**（Tool Layer）:

   用户最先接触到的部分，包括关卡编辑器、角色编辑器、动画编辑器等。

   是游戏引擎最直观的第一印象。

2. **功能层**（Function Layer）:

   让游戏世界“可以被看见、可以动起来、可以玩起来”的核心服务。

   包括渲染（Rendering）、动画（Animation）、物理（Physics）、脚本（Script）、AI（人工智能）、用户界面（UI）等。

3. **资源层**（Resource Layer）:

   位于功能层之下，负责管理、加载和处理所有游戏数据文件。

   这些数据文件（如从3ds Max, Maya, Photoshop导出的模型、贴图、声音等）被称为**资源（Resource）**。

4. **核心层**（Core Layer）:

   一个功能强大的“工具箱”，为上层提供底层的基础服务。

   包括内存管理、数据结构（容器）、数学库（线性代数、坐标变换）、线程管理等。

5. **平台层**（Platform Layer）:

   最底层，负责屏蔽不同硬件和软件平台的差异性。

   处理不同操作系统（Windows, Mac, Linux）、图形API（DirectX, Metal, Vulkan）、CPU架构（x86, ARM）和输入设备（键盘、鼠标、手柄）等。

这构成了一个基础的 **5层架构**。此外，许多引擎还会集成**第三方代码库**（如 Havok Physics, SpeedTree），形成“5+1”的生态系统。

---

3. 资源层（Resource Layer）详解 *Content-[04:01]

资源层是游戏引擎的“弹药库”，处理从原始数据到引擎内部高效数据的转换和管理。

3.1 从 Resource 到 Asset

问题： 游戏引擎不能直接读取原始的 `.max`，`.maya`，`.psd` 等格式。这些文件体积大，包含大量无效编辑器信息，且数据排列方式不利于实时渲染。

转换过程： 将原始资源（Resource）转换为引擎专用的高效数据格式，称为**资产（Asset）**。

例子： 一张 `.jpg` 或 `.png` 的贴图会被转换为 `.dds`（DirectDraw Surface）格式。DDS格式按照16字节对齐，数据块紧凑排列，可以直接被GPU读取为纹理。

3.2 资产的关键概念

复合资产（Composed Asset）： 一个资产可以引用其他资产，定义它们之间的关系。例如，一个“角色”资产会通过一个XML/JSON配置文件，声明它引用的网格（Mesh）、贴图（Texture）和动画（Animation）资产。

GUID（全局唯一标识符，Globally Unique Identifier）： 现代引擎中，为每个资产分配一个唯一的ID（就像人的身份证号），而不是依赖文件路径。这解决了文件重命名或移动路径导致引用失效的问题。引擎通过查找GUID来加载资产，而不是文件路径。

运行时资产管理器（Runtime Asset Manager）： 游戏运行时，一个核心管理器负责管理所有已加载资产的生命周期。它通常使用 **Handle 系统**（类似邮箱的钥匙，不直接持有资产地址，而是通过一个抽象句柄来间接访问），来处理资产的加载、卸载和引用关系。

延迟加载（Lazy Loading）： 引擎只加载当前关卡或视野内需要的资产，并随着玩家推进动态加载和卸载新资产。例如，在开放世界游戏中，远处的贴图刚开始可能很模糊（低分辨率版本），当玩家走近时再加载高分辨率版本，这就是延迟加载的体现。

关键结论： 资源层是引擎的核心管理层，许多引擎开发的第一步往往是定义数据格式和Schema，而非编写图形功能。

---

4. 功能层（Function Layer）详解：Tick与神兽 *Content-[21:52]

功能层是游戏世界的核心驱动力，其核心是 **Tick** 机制。

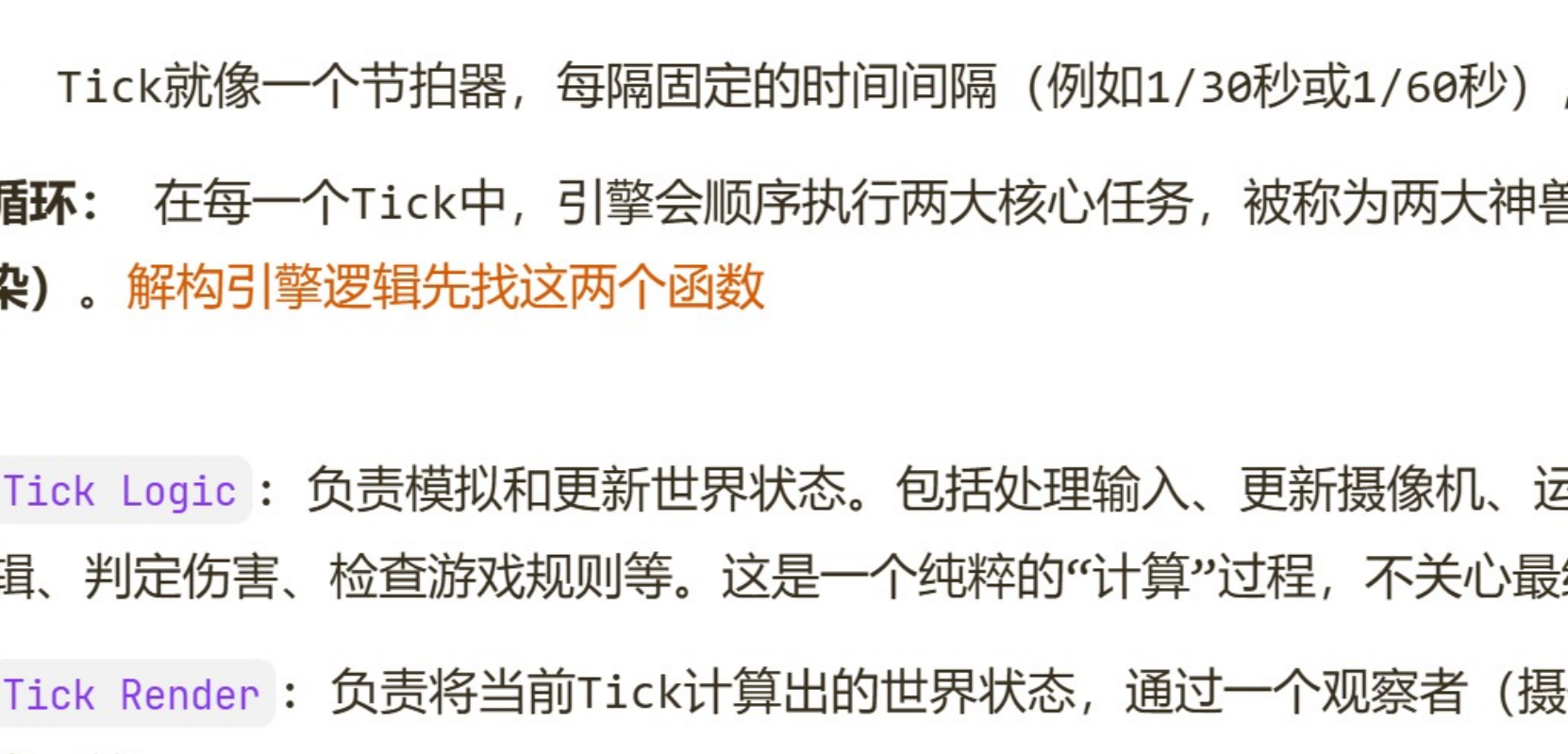
4.1 Tick机制

概念： Tick就像一个节拍器，每隔固定的时间间隔（例如1/30秒或1/60秒），推动整个游戏世界前进一小步。

核心循环： 在每一个Tick中，引擎会顺序执行两大核心任务，被称为两大神兽： **Logic（逻辑）** 和 **Render（渲染）**。解构引擎逻辑先找这两个函数

`Tick Logic`：负责模拟和更新世界状态。包括处理输入、更新摄像机、运行物理碰撞、播放动画、执行AI逻辑、判定伤害、检查游戏规则等。这是一个纯粹的“计算”过程，不关心最终是否被看到。

`Tick Render`：负责将当前Tick计算出的世界状态，通过一个观察者（摄像机）的视角，绘制成一张二维图像（帧）。



```
void tickMain(float delta_time)
{
    while (!exit_flag)
    {
        tickLogic(delta_time);
        tickRender(delta_time);
    }
}
```

```
void tickLogic(float delta_time)
{
    tickCamera(delta_time);
    tickMotor(delta_time);
    tickController(delta_time);
    tickAnimation(delta_time);
    tickPhysics(delta_time);
    /*...*/
}
```

```
void tickRender(float delta_time)
{
    tickRenderCamera();
    culling();
    rendering();
    postprocess();
    present();
}
```

形象比喻:

1. **逻辑（Logic）**： 上帝在每一普朗克时间，先将整个宇宙的物理规律计算一遍，确定万物当下的状态（位置、姿态、生命值等）。该过程不依赖观察者。
2. **渲染（Render）**： 观察者（小明）出现，从自己的视角观察这个世界，生成一幅画面。这个过程依赖于观察者的位置和视角。

重要原则： 逻辑（Logic）和渲染（Render）的代码必须**严格分离**。混合在一起是初学者常见错误，将导致代码混乱、难以维护和调试。

4.2 功能层的扩展：多核并行架构

单线程时代： 最早的引擎用一个线程从头跑到尾，效率低下。

双线程时代： 最简单的并行优化是将 `Tick Logic` 和 `Tick Render` 分到两个线程。将渲染和逻辑分离到两个线程中。

Job System（任务系统）： 这是未来引擎架构的方向。引擎不再区分固定的大线程，而是将所有任务（物理、动画、AI、裁剪等）打散成无数个独立的、最小的原子任务（Job），然后由系统根据CPU核心数动态调度，尽可能占满所有核心。

打开一个游戏，打开cpu的profiling工具，看小窗口是否吃的很满

挑战： 任务之间存在**依赖关系（Dependency）**。例如，粒子特效的计算必须等动画系统计算出“手”的位置之后才能进行。如何高效、无错地管理这种复杂的依赖关系，是设计Job System的核心难点。

---

5. 核心层（Core Layer）详解 *Content-[31:31]

核心层是引擎的“瑞士军刀”，为上层的所有功能提供高效的基础服务。

5.1 数学库（Math Library）

基础： 主要用到大学水平的线性代数（向量、矩阵、坐标变换）。

关键：效率！ 游戏是实时应用，对数学运算效率要求极高。

优化技巧： 使用近似算法，如 John Carmack 著名的 `1/sqrt(x)` 算法，通过牛顿迭代法快速逼近，不追求绝对精确，但性能远超标准库函数。

SIMD（单指令多数据流，Single Instruction, Multiple Data）： 现代CPU提供了SIMD指令集，**一条指令可以同时对多个数据（如一个4D向量的四个分量）进行运算**。游戏引擎的数学库会大量使用SSE（Streaming SIMD Extensions）等指令集加速向量运算。

结论： 游戏引擎的数学库因其对效率的极致追求，通常需要手写，不能直接依赖高级语言的标准数学库。

5.2 数据结构（Data Structures）与内存管理

为什么不用C++ STL库? 标准库的容器（如 `std::vector`）在频繁的增删操作中会产生大量内存碎片，控制内存分配的行为，并且其扩容策略（如2倍扩容）会导致不可预测的内存消耗。

核心层的目标： 创建一套**无内存碎片、访问效率极高**的自定义数据结构。

内存管理三原则（图灵机视角）：

1. **空间局部性（Spatial Locality）**： 将频繁访问的数据尽可能放在一起（连续内存）。
2. **时间局部性 / 顺序访问（Temporal Locality / Sequential Access）**： 访问数据时，尽量沿着内存地址的顺序进行（如 `123456`，而非 `179356`）。
3. **批量处理（Batching）**： 创建或销毁数据时，一次申请/释放一大块内存，而不是频繁地单个操作。

实现方式： 引擎会预先向操作系统申请一大块内存，然后自己负责管理这块内存的分配和回收，以实现上述原则。

地位： 核心层的代码质量要求最高，通常由引擎团队中最资深的程序员编写并严格维护，轻易不做改动。core代码

---

6. 平台层（Platform Layer）详解 *Content-[41:40]

平台层是引擎的“翻译官”，其核心目标是实现**平台无关性（Platform Agnosticism）**，让上层代码在编写时无需关心底层硬件和操作系统的差异。

6.1 需要屏蔽的差异

简单差异： 文件路径格式（Windows正斜杠 `\` vs Mac反斜杠 `/`）。

复杂差异： 图形API差异（DirectX 11/12 vs Metal vs Vulkan vs OpenGL ES）。

底层差异： CPU架构差异，如PlayStation 3的自研SPU协处理器，或者现代ARM CPU的大小核架构。

6.2 核心抽象：RHI

RHI（渲染硬件接口，Render Hardware Interface）：是平台层中最重要的抽象之一。它会定义一套统一的图形API接口。

工作方式： 功能层的程序员调用统一的RHI接口来绘制图形，而RHI的具体实现在编译时会根据目标平台（PC、Mac、PS5）选择对应的底层API（DX12, Metal, ...）进行翻译。

难点： 不同版本的同一种API（如DX11 vs DX12）其设计哲学和管线差异巨大，简单封装可能导致性能下降，必须根据底层API特性进行精细优化。

6.3 平台层的重要性

引擎水平的分水岭： 一个好的平台层能让引擎高效地运行在多个平台，是一个引擎能力强弱的重要标志。

---

7. 工具层（Tool Layer）详解 *Content-[46:02]

工具层是游戏内容的“创造工厂”，它解放了艺术家和游戏设计师的创造力，是真正的生产力工具。

7.1 核心编辑器

以关卡编辑器（Level Editor）为中心： 形成一系列配套编辑器，如蓝图逻辑编辑器、材质编辑器、动画编辑器等。

目的： 提供所见即所得（WYSIWYG）的编辑环境。例如，引擎内的材质编辑器必须保证其预览效果与游戏运行时完全一致。

7.2 数据管线（Asset Conditioning Pipeline）

来源： 工具层不仅包含自研的编辑器，还集成了大量第三方DCC工具（Digital Content Creation，如3ds Max, Maya, Photoshop, Houdini）。

管线： 所有DCC工具和自研编辑器产生的数据，通过一个统一的“资产处理管线”（Asset Conditioning Pipeline），经过导入、导出、格式转换等步骤，最终变为引擎内部统一的**Asset**格式。

7.3 工具层的特点

代码量大： 工具层的代码量通常比引擎的Runtime部分还要大，其维护成本和复杂度也更高。

开发灵活： 工具层不像Runtime追求极致的运行效率，它更追求开发效率。可以使用C#（WPF），HTML5，Python等多种语言和框架进行开发。

生态体系： 几十种不同的工具之间需要保证数据流畅通无阻，这是一个巨大的工程挑战，通常需要一个统一的数据格式（如FBX, USD）或自研格式来解决。

---

8. 分层架构的核心逻辑与意义 *Content-[50:27]

游戏引擎的分层架构，本质上是一种**复杂系统管理**的艺术。

8.1 为何要分层

解耦复杂性（Decoupling Complexity）： 将巨大的系统复杂度分解到不同的层次。每一层只关注自己的职责，使用下层提供的服务，并向上一层提供服务。这类似于经典的系统科学理论—**封装（Encapsulation）**：原子构成篮球，篮球构成比赛，每个层级都是一个独立的封装。

控制变量法： 开发和调试时，可以专注于某一层的问题。

8.2 分层架构的原则

单向依赖： 只允许**上层调用下层的功能**，绝对禁止下层反向调用上层。这是架构的核心原则。

稳定性梯度：

越底层越稳定： 平台层、核心层的代码一旦写好就钉死了，几年甚至十几年不轻易改动。

越上层越灵活： 功能层和工具层变化频繁，每天都会有新的逻辑和工具被添加。

8.3 对开发者的指导意义

思考归属： 在实现任何新功能前，首先思考：“这个功能属于哪一层？应该在哪一层写代码？”

系统性训练： 能够清晰划分功能层次，是现代游戏引擎工程师的基本素养。

---

9. 课程实践与总结 *Content-[54:11]

9.1 课程小引擎：Pilot

命名缘由： Pilot意为“领航员”，希望这个微小的引擎能引领学生进入游戏引擎的海洋。

代码量： 约15,100行C/C++代码（Runtime约1.3万行，编辑器约2千行）。

架构： 严格遵循课程讲述的分层架构，结构非常工整。

功能： 支持基本的编辑器、渲染（包括反射）、简单的碰撞胶囊、一键Play等。后续课程将逐步开放动画、物理、❓ECS等系统，最终目标是实现一个多人在线对战游戏。

9.2 本节课核心总结（三件事）

1. **游戏引擎是分层架构的**，记住 **五层**:

   平台层（Platform）

   核心层（Core）

   资源层（Resource）

   功能层（Function）

   工具层（Tool）

2. **架构的核心逻辑**:

   越底层越稳定，代码质量要求越高。

   越上层越开放灵活，以适应不同产品需求。

3. **游戏世界的核心驱动力**:

   所有引擎都通过一个 **Tick** 函数来驱动世界运行。

   找到它，你将理解引擎的运行逻辑。

9.3 Q&A（精选）

Q: 不同引擎（如UE4, Unity）的分层结构一样吗?

A: 大同小异。Platform和Core层基本都有。Resource层也普遍存在。Function层的划分方法可能差别较大，但基础逻辑（抽Tick/Update）是一样的。

Q: 为什么游戏引擎不用标准的C++ STL库?

A: STL库的内存管理策略在高频操作下会产生大量内存碎片，且行为不可控。游戏引擎对性能极其敏感，需要手写定制化的、零碎片的容器。